

# Documentación de código React + Tailwind para VS Code

## Objetivo

Que VS Code entienda tu código y te ayude a:

- Autocompletar funciones y sus parámetros
  - Mostrar descripciones al hacer hover
  - Detectar errores de tipo antes de ejecutar
  - Sugerir correcciones automáticas
- 

## 1. Documentar componentes con JSDoc

JSDoc es la forma estándar de documentar JavaScript/React. VS Code lo entiende perfectamente.

### Ejemplo básico de componente

```
jsx
/**
 * Componente de botón personalizado con estilos de Tailwind
 *
 * @param {Object} props - Las propiedades del componente
 * @param {string} props.texto - El texto que se muestra en el botón
 * @param {() => void} props.onClick - Función que se ejecuta al hacer clic
 * @param {string} [props.color="blue"] - Color del botón (blue, red, green)
 * @returns {JSX.Element}
 */
const Boton = ({ texto, onClick, color = "blue" }) => {
  const colores = {
    blue: "bg-blue-500 hover:bg-blue-700",
    red: "bg-red-500 hover:bg-red-700",
    green: "bg-green-500 hover:bg-green-700"
  }

  return (
    <button
      onClick={onClick}
      className={` ${colores[color]} text-white font-bold py-2 px-4 rounded`}
    >
      {texto}
    </button>
  )
}
```

### ¿Qué logramos?

- Al escribir <Boton texto=" VS Code sugerirá que falta onClick
  - Al hacer hover sobre <Boton> aparecerá la descripción completa
  - Al pasar el mouse sobre color mostrará que es opcional y su valor por defecto
-

## 2. Documentar props con PropTypes (Alternativa)

Si prefieren validación en tiempo de ejecución:

```
jsx
import PropTypes from 'prop-types'

const Boton = ({ texto, onClick, color = "blue" }) => {
  // ... código
}

Boton.propTypes = {
  /** El texto que se muestra en el botón */
  texto: PropTypes.string.isRequired,
  /** Función que se ejecuta al hacer clic */
  onClick: PropTypes.func.isRequired,
  /** Color del botón (blue, red, green) */
  color: PropTypes.oneOf(['blue', 'red', 'green'])
}
```

## 3. Documentar hooks personalizados

```
jsx
/**
 * Hook para manejar el estado de un formulario
 *
 * @param {Object} initialState - Estado inicial del formulario
 * @returns {Object} - { values, handleChange, resetForm }
 *
 * @example
 * const { values, handleChange } = useForm({ nombre: "", email: "" })
 */
const useForm = (initialState) => {
  const [values, setValues] = useState(initialState)

  const handleChange = (e) => {
    setValues({ ...values, [e.target.name]: e.target.value })
  }

  const resetForm = () => setValues(initialState)

  return { values, handleChange, resetForm }
}
```

## 4. Documentar funciones de utilidad

```
jsx
/**
 * Formatea un número como moneda argentina
 *
 * @param {number} monto - El número a formatear
 * @param {boolean} [incluirSimbolo=true] - Si se incluye el símbolo $
 * @returns {string} Monto formateado (ej: "$ 1.234,56")
 */
```

```

*
* @example
* formatMoney(1234.56) // "$ 1.234,56"
* formatMoney(1234.56, false) // "1.234,56"
*/
const formatMoney = (monto, incluirSimbolo = true) => {
  const formateado = monto.toLocaleString('es-AR', {
    minimumFractionDigits: 2,
    maximumFractionDigits: 2
  })

  return incluirSimbolo ? `$ ${formateado}` : formateado
}

```

## 5. Configurar VS Code para mejorar la experiencia

### Extensiones recomendadas

| Extensión                              | Para qué sirve                       |
|----------------------------------------|--------------------------------------|
| JavaScript and TypeScript Nightly      | Mejor soporte de JS moderno          |
| Tailwind CSS IntelliSense              | Autocompletado de clases Tailwind    |
| ES7+ React/Redux/React-Native snippets | Atajos para React (rafce, rfc, etc.) |
| Prettier                               | Formateo automático                  |
| Error Lens                             | Muestra errores inline               |

### Configuración recomendada (settings.json)

```

json
{
  "editor.quickSuggestions": {
    "strings": true
  },
  "javascript.suggest.completeFunctionCalls": true,
  "editor.parameterHints.enabled": true,
  "editor.hover.enabled": true,
  "tailwindCSS.includeLanguages": {
    "javascript": "javascript",
    "javascriptreact": "javascriptreact"
  },
  "tailwindCSS.emmetCompletions": true
}

```

## 6. Estructura de documentación por archivo

```
jsx
/**
 * @fileoverview Componentes de formulario para inscripción a eventos
 * @author Nombre del alumno
 * @version 1.0.0
 *
 * Este archivo contiene los componentes principales del formulario:
 * - FormularioInscripcion: formulario principal
 * - CampoPersonalizado: wrapper de inputs con estilos
 * - BotonEnvio: botón personalizado
 */
```

## 7. Buenas prácticas

Bueno ❤️

Malo ❌

---

@param {string} nombre - Descripción clara

@param {string} nombre (sin descripción)

---

@returns {JSX.Element} Descripción del componente

@returns (vacío)

---

@example con código real

@example con pseudo-código

---

@deprecated para funciones obsoletas

Borrar funciones sin avisar

---

## 8. Atajos útiles en VS Code

Atajo

Función

---

/\*\* + Enter

Genera JSDoc automáticamente

---

Ctrl + Espacio

Activar autocompletado manual

---

F12

Ir a definición de la función

---

Shift + F12

Ver todas las referencias

---

Ctrl + K Ctrl + I

Mostrar hover info del elemento

---

## 9. Ejemplo completo de componente bien documentado

```
jsx
/**
 * @fileoverview Componente de tarjeta de producto con imagen y precio
 */

import { useState } from 'react'
import PropTypes from 'prop-types'

/**
 * Componente que muestra un producto con opción de agregar al carrito
 *
 * @component
 * @param {Object} props
 * @param {string} props.nombre - Nombre del producto
 * @param {number} props.precio - Precio del producto
 * @param {string} props.imagenUrl - URL de la imagen del producto
 * @param {() => void} props.onAgregar - Función al agregar al carrito
 * @returns {JSX.Element}
 *
 * @example
 * <Producto
 *   nombre="Remera Oversize"
 *   precio={12500}
 *   imagenUrl="/images/remera.jpg"
 *   onAgregar={() => console.log('Agregado!')}
 * />
 */
const Producto = ({ nombre, precio, imagenUrl, onAgregar }) => {
  const [cantidad, setCantidad] = useState(1)

  return (
    <div className="max-w-sm rounded overflow-hidden shadow-lg bg-white">
      <img className="w-full h-48 object-cover" src={imagenUrl} alt={nombre} />
      <div className="px-6 py-4">
        <h3 className="font-bold text-xl mb-2">{nombre}</h3>
        <p className="text-gray-700 text-base">${precio.toLocaleString('es-AR')}</p>

        <div className="mt-4 flex items-center gap-2">
          <input
            type="number"
            min="1"
            value={cantidad}
            onChange={(e) => setCantidad(Number(e.target.value))}
            className="w-20 px-2 py-1 border rounded"
          />
          <button
            onClick={() => onAgregar(cantidad)}
            className="bg-blue-600 text-white px-4 py-1 rounded hover:bg-blue-700 transition"
          >
            Agregar
          </button>
        </div>
      </div>
    </div>
  )
}
```

```
        </div>
    </div>
)
}

Producto.propTypes = {
  nombre: PropTypes.string.isRequired,
  precio: PropTypes.number.isRequired,
  imageUrl: PropTypes.string.isRequired,
  onAgregar: PropTypes.func.isRequired,
}

export default Producto
```

## Conclusión

Con JSDoc y PropTypes, VS Code se convierte en un asistente inteligente que:

- Autocompleta funciones y parámetros
- Advirte sobre tipos incorrectos
- Muestra documentación al hacer hover
- Facilita el mantenimiento del código

**Regla de oro: Si un componente o función va a ser usado por más de una persona (o por vos mismo dentro de un mes), documentalo como si fuera para un desconocido.**